

DW03_bictr_dcnto

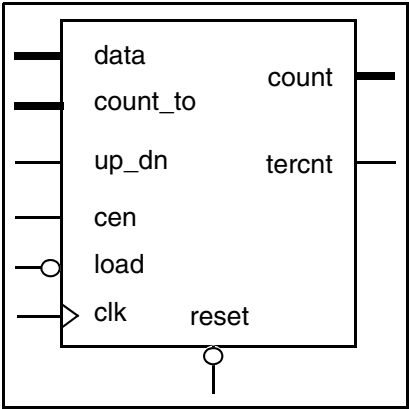
Up/Down Binary Counter with Dynamic Count-to Flag

Version, STAR, and myDesignWare Subscriptions: [IP Directory](#)

Features and Benefits

- Parameterized word length
- Terminal count flag for count-to comparison
- Pin-programmable count-to value
- Up/down count control
- Asynchronous reset
- Synchronous counter load
- Synchronous count enable
- Includes a low-power implementation that has power benefits from minPower optimization (for details, see [Table 1-3](#) on page 2)

Revision History



Description

DW03_bictr_dcnto is a general-purpose up/down counter with dynamic count-to logic.

Table 1-1 Pin Description

Pin Name	Width	Direction	Function
data	<i>width</i>	Input	Counter load input
count_to	<i>width</i>	Input	Count compare input
up_dn	1	Input	High for count up and low for count down
load	1	Input	Enable data load to counter, active low
cen	1	Input	Count enable, active high
clk	1	Input	Clock
reset	1	Input	Counter reset, active low
count	<i>width</i>	Output	Output count bus
tercnt	1	Output	Terminal count flag, active high

Table 1-2 Parameter Description

Parameter	Values	Description
width	≥ 1	Width of data input bus

Table 1-3 Synthesis Implementations

Implementation Name	Function	License Feature Required
str	Synthesis model	DesignWare
lpwr ^{ab}	Low Power Synthesis model	- DesignWare (P-2019.03 and later) - DesignWare-LP (before P-2019.03)

- a. Requires that you enable minPower; for details, see [“Enabling minPower”](#) on page 5.
When minPower is enabled, the lpwr implementation is always chosen during synthesis.
- b. Effectiveness of low power design depends on the use of the `-gate_clock` option to `compile_ultra` command

Table 1-4 Simulation Models

Model	Function
DW03.DW03_BICTR_DCNT0_CFG_SIM	Design unit name for VHDL simulation
dw/dw03/src/DW03_bictr_dcnto_sim.vhd	VHDL simulation model source code
dw/sim_ver/DW03_bictr_dcnto.v	Verilog simulation model source code

Table 1-5 Counter Operation Truth Table

reset	load	cen	up_dn	Operation
0	X	X	X	Reset
1	0	X	X	Load
1	1	0	X	Standby
1	1	1	0	Count down
1	1	1	1	Count up

When the count value equals the value on the `count_to` pin, the signal `tercnt` (terminal count) is asserted (high). The signal `tercnt` can be connected to `load` through an inversion to synchronously reset the counter to a predefined value on the input pin of the data bus, `data`.

The counter is *width* bits wide and has 2^{width} states from “000...0” to “111...1”. The counter is clocked on the positive edge of `clk`.

The `reset`, active low, provides for an asynchronous reset of the counter to “000...0”. If the reset pin is connected to ‘1’, then the reset logic is not synthesized, resulting in a smaller and faster counter.

The `count_to` is an input bus that ranges from 0 to `width-1`. When the counter output, `count`, equals `count_to`, `tercnt` goes high for one clock cycle.

The `up_dn` input controls whether the counter counts up (`up_dn` is high) or down (`up_dn` is low), starting on the next positive edge of `clk`.

The counter is loaded with `data` by asserting `load` (low) and applying data to `data`. The data load operation is synchronous with respect to the positive edge of `clk`.

The count enable pin, `cen`, is active high. When `cen` is high, the counter is active. When `cen` is low, the counter is disabled, and `count` remains at the same value.

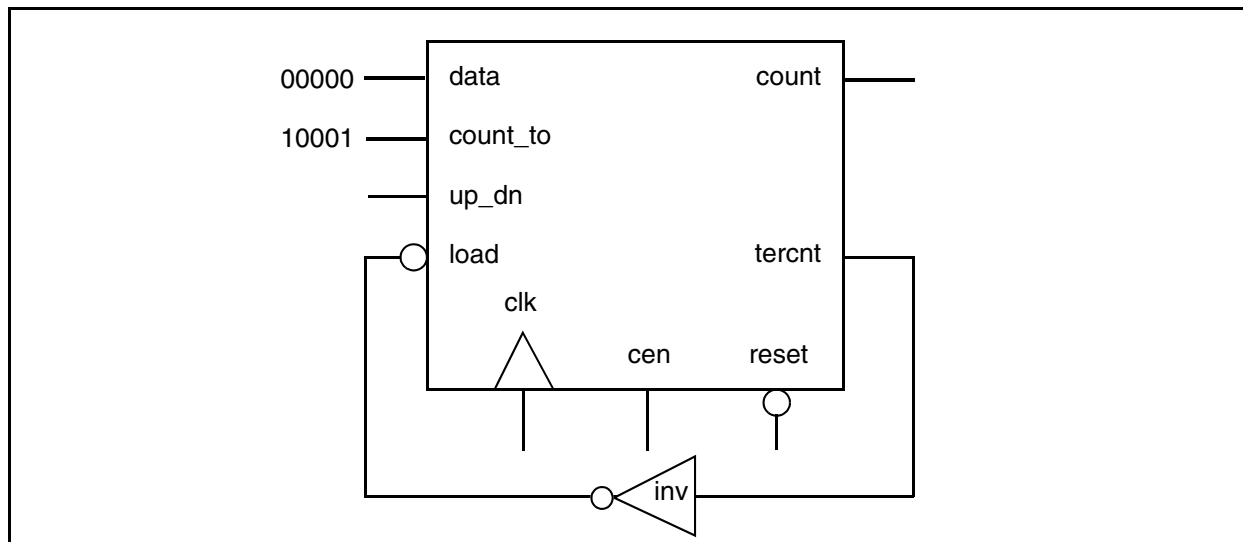
Application Example

An example application of DW03_bictr_dcnto is to count from 0 to 17, repeatedly. This is done by:

1. Connecting the terminal count output signal, `tercnt`, to the load input.
2. Setting the data input to the start of the count sequence (for example, “00000”).
3. Connecting the `count_to` input to the end of the count sequence (for example, “10001”).

The count output then cycles through the states 00000 (`data`) to 10001 (`count_to`); see [Figure 1-1](#).

Figure 1-1 Counter Application: width = 5



Timing Diagrams

Figure 1-2 on page 4 and Figure 1-3 on page 4 show various timing diagrams for DW03_bictr_dcnto.

Figure 1-2 Functional Operation: reset, load, and count_to

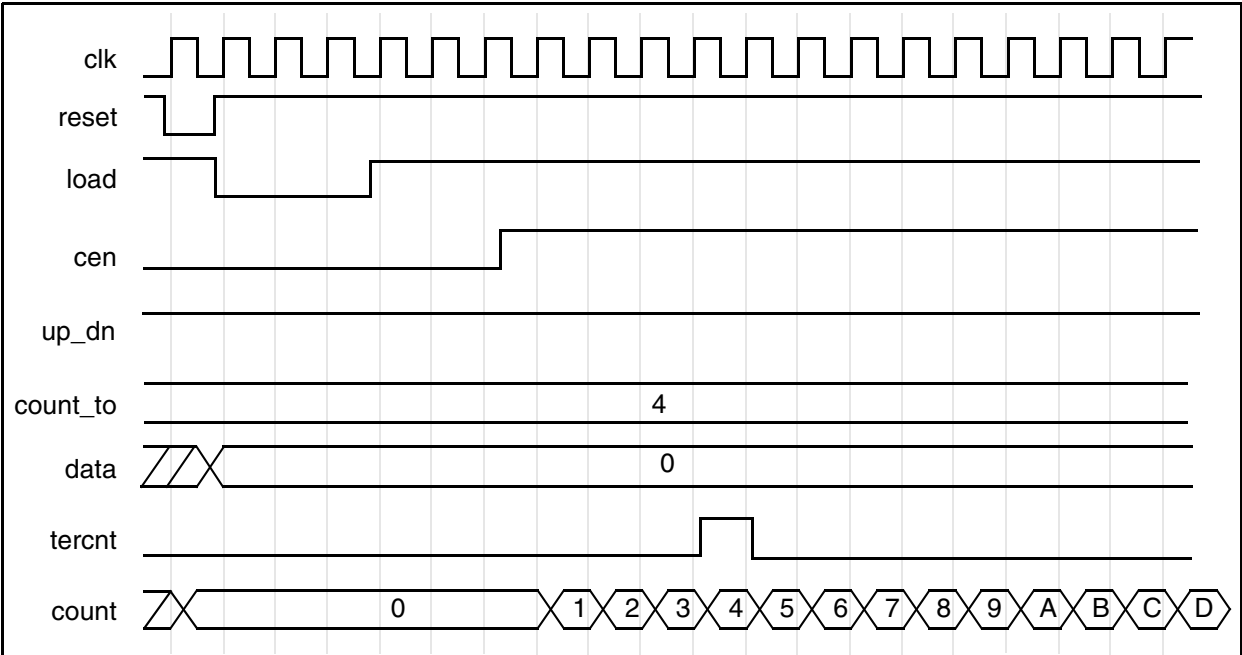
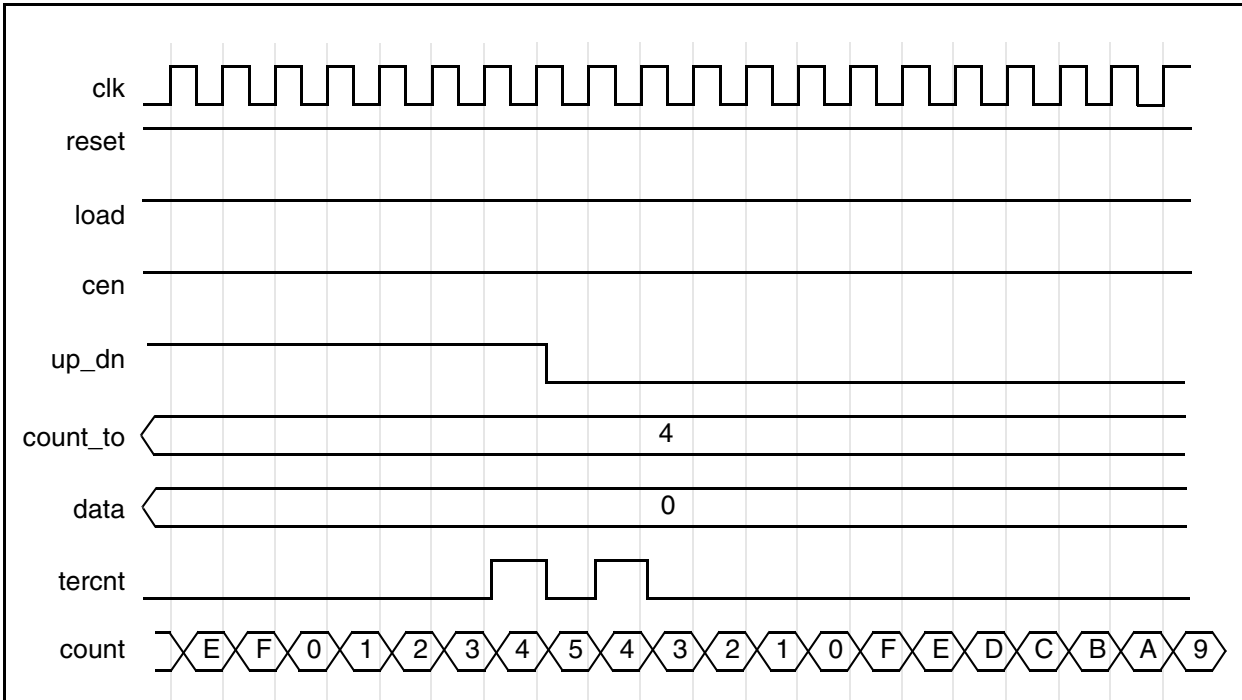


Figure 1-3 Functional Operation: Up and Down Counting, and Count-to Sequence



Enabling minPower

You can instantiate this component without enabling minPower, but to achieve power savings from the low-power implementation “lpwr” (see [Table 1-3](#) on page 2), you must enable minPower optimization, as follows:

- Design Compiler

- Version P-2019.03 and later:

- ```
set power_enable_minpower true
```

- Before version P-2019.03 (requires the DesignWare-LP license feature):

- ```
set synthetic_library {dw_foundation.sldb dw_minpower.sldb}  
set link_library {* $target_library $synthetic_library}
```

- Fusion Compiler

Optimization for minPower is enabled as part of the total_power metric setting. To enable the total_power metric, use the following:

```
set_qor_strategy -stage synthesis -metric total_power
```

Related Topics

- [Logic – Sequential Overview](#)
- [DesignWare Building Block IP User Guide](#)

HDL Usage Through Component Instantiation - VHDL

```
library IEEE,DWARE;
use IEEE.std_logic_1164.all;
use DWARE.DWpackages.all;
use DWARE.DW_foundation_comp.all;

entity DW03_bictr_dcnto_inst is
  generic (inst_width : POSITIVE := 8);
  port (inst_data      : in std_logic_vector(inst_width-1 downto 0);
        inst_count_to  : in std_logic_vector(inst_width-1 downto 0);
        inst_up_dn     : in std_logic;
        inst_load       : in std_logic;
        inst_cen        : in std_logic;
        inst_clk        : in std_logic;
        inst_reset      : in std_logic;
        count_inst      : out std_logic_vector(inst_width-1 downto 0);
        tercnt_inst     : out std_logic);
end DW03_bictr_dcnto_inst;

architecture inst of DW03_bictr_dcnto_inst is
begin

  -- Instance of DW03_bictr_dcnto
  U1 : DW03_bictr_dcnto
    generic map ( width => inst_width )
    port map ( data => inst_data, count_to => inst_count_to,
              up_dn => inst_up_dn, load => inst_load, cen => inst_cen,
              clk => inst_clk, reset => inst_reset, count => count_inst,
              tercnt => tercnt_inst );

  end inst;

  -- pragma translate_off
  configuration DW03_bictr_dcnto_inst_cfg_inst of DW03_bictr_dcnto_inst is
    for inst
      end for; -- inst
  end DW03_bictr_dcnto_inst_cfg_inst;
  -- pragma translate_on
```

HDL Usage Through Component Instantiation - Verilog

```
module DW03_bictr_dcnto_inst( inst_data, inst_count_to, inst_up_dn,
                             inst_load, inst_cen, inst_clk, inst_reset,
                             count_inst, tercnt_inst );

    parameter width = 8;

    input [width-1 : 0] inst_data;
    input [width-1 : 0] inst_count_to;
    input inst_up_dn;
    input inst_load;
    input inst_cen;
    input inst_clk;
    input inst_reset;
    output [width-1 : 0] count_inst;
    output tercnt_inst;

    // Instance of DW03_bictr_dcnto
    DW03_bictr_dcnto #(width)
        U1 ( .data(inst_data), .count_to(inst_count_to), .up_dn(inst_up_dn),
            .load(inst_load), .cen(inst_cen), .clk(inst_clk),
            .reset(inst_reset), .count(count_inst), .tercnt(tercnt_inst) );
endmodule
```

Revision History

For notes about this release, see the [DesignWare Building Block IP Release Notes](#).

For lists of both known and fixed issues for this component, refer to the [STAR report](#).

For a version of this datasheet with visible change bars, click [here](#).

Date	Release	Updates
September 2024	DWBB_202409.0	■ Updated date and version; no technical changes
December 2023	DWBB_202312.0	■ Updated date and version; no technical changes
March 2019	DWBB_201903.0	■ Clarified license requirements in Table 1-3 on page 2 ■ Added “ Enabling minPower ” on page 5
January 2019	DWBB_201806.5	■ Updated example in “ HDL Usage Through Component Instantiation - VHDL ” on page 6 ■ Added this Revision History table and the document links on this page

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

